

Coding & Solution

Date	06 July 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

- 1 Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
- 2 The solution should address the core problem statement and implement the key functional requirements.
- 3 Include all source files, configuration files, and setup instructions needed to run the application.
- 4 Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	Not provided — no GitHub/GitLab or hosted link was included with the submitted project files.
Programming Language(s)	Python
Framework(s) Used	Flask (web app); pandas, scikit-learn, XGBoost, joblib (data processing & modelling)

Field	Details
Key Features Implemented	• Applicant data entry web form (index.html) capturing demographic, income, employment, housing and contact details. • Real-time approval/rejection prediction via a Flask /predict endpoint. • Feature vector engineering (build_feature_vector) — one-hot encoding of income type, family status, occupation and housing type, plus derived fields such as DEPENDENTS_RATIO. • Tuned classification threshold (0.29, cross-validation selected) applied instead of the default 0.5. • Approval-probability gauge visualization and a plain-language model performance disclosure on the result page. • Vintage (months-on-book) based data cleaning/labelling pipeline to reduce label noise in the training data. • XGBoost model training with cross-validated hyperparameter tuning (epic4_model_development_v3.py), exported to a version-portable native format.
Pending / Incomplete Features	• User authentication / login and role-based access (customer vs. administrator) — listed as a requirement but not implemented in the current app.py. • No automated tests (unit/integration) included in the submitted files. • No error handling (try/except) around model loading, file I/O, or form parsing — invalid input will currently raise an unhandled exception rather than a friendly error. • No HTTPS/deployment configuration or cloud (e.g., IBM Watson) integration present in the code, despite being mentioned in the requirements doc. • Analysis notebook (Epic2 Visualisation & Analysis) is present but not wired into the running application.
Setup / Run Instructions	1. Install dependencies: pip install -r requirements.txt (flask, pandas, scikit-learn, xgboost, joblib). 2. Ensure the model/ folder (best_model_v3_tuned.json, feature_columns_v3.pkl, model_metadata_v3.json) is present alongside app.py. 3. Run: python app.py 4. Open http://localhost:5000 in a browser, fill in the applicant form and submit to view the prediction.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Partial — app.py is modular (build_feature_vector, index, predict); the preprocessing and model-training scripts (epic3, epic4) are written as top-to-bottom procedural scripts rather than functions/classes.
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes — module-level docstrings explain rationale (e.g., vintage-analysis fix, native model format), and app.py has inline comments on non-obvious logic.
4	Error handling is implemented for critical operations	No — no try/except blocks were found around model loading, file I/O, or form input parsing in app.py, epic3, or epic4.
5	The application runs without critical errors	Likely, based on code review — not independently executed/tested in this environment; requires the model/ artifacts and dataset paths to be present.
6	Code is committed to a version control repository	Cannot verify — no repository link or .git metadata was included in the submitted files.

Additional Notes / Comments:

The core prediction pipeline (form → feature engineering → tuned XGBoost model → result page with transparency disclosure) is fully implemented and reflects thoughtful modelling choices (vintage-based labelling, tuned threshold, honest precision/recall disclosure to the user). The main gaps against the project's own Solution Requirements doc are: no authentication/authorization layer, no automated tests, no error handling for bad input or missing model files, and no repository link was supplied for this review.